

 Building a Streamlit Chat App with Ollama and OpenAI	1
💡 Why this matters	1
🔧 The Code	2
▶ Running the App	4
☀ Example Use Cases	5
 Takeaway	6

Building a Streamlit Chat App with Ollama and OpenAI

Large Language Models (LLMs) are no longer limited to the cloud. With tools like **Ollama**, you can now run powerful open-source models locally on your laptop, while still having the option to call cloud providers like **OpenAI** when needed.

In this blog, we'll walk through how to create a **Streamlit app** that lets you **choose between local Ollama models or cloud-based OpenAI models** — all in one simple UI.

Why this matters

- **Flexibility:** Switch between local (privacy-friendly, free) and cloud (state-of-the-art, scalable) models.
- **Experimentation:** Try open-source models like `llama3.1:8b` or `gpt-oss-20b` on your Mac/PC without GPU clusters.

- **Ease of use:** A simple Streamlit interface makes this accessible to developers and non-developers alike.
-

The Code

Here's the complete Streamlit app:

```
import streamlit as st
import requests
from openai import OpenAI

# ---- Helper: Ollama ----
def chat_with_ollama(messages, model="llama3.1:8b"):
    prompt = ""
    for m in messages:
        role = m["role"].capitalize()
        prompt += f"{role}: {m['content']}\n"
    prompt += "Assistant:"

    resp = requests.post(
        "http://127.0.0.1:11434/api/generate",
        json={"model": model, "prompt": prompt, "stream": False},
        timeout=60
    )
    resp.raise_for_status()
    data = resp.json()
    return data["response"]

# ---- Helper: OpenAI ----
def chat_with_openai(messages, model="gpt-3.5-turbo", api_key=None):
    client = OpenAI(api_key=api_key)
    resp = client.chat.completions.create(
        model=model,
        messages=messages
    )
    return resp.choices[0].message.content

# ---- Streamlit UI ----
```

```

st.title("💬 Chat with Ollama or OpenAI")

provider = st.radio("Choose provider", ["Ollama (local)", "OpenAI (cloud)"])

if provider == "OpenAI (cloud)":
    openai_api_key = st.text_input("Enter OpenAI API Key", type="password")
    openai_model = st.selectbox("Select OpenAI model", ["gpt-3.5-turbo", "gpt-4"])
else:
    ollama_model = st.selectbox("Select Ollama model", ["llama3.1:8b", "gpt-oss-20b"])

user_input = st.text_area("Your message")

if st.button("Send"):
    if provider == "Ollama (local)":
        messages = [
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": user_input}
        ]
        reply = chat_with_ollama(messages, model=ollama_model)
        st.markdown(f"**Assistant:** {reply}")

    elif provider == "OpenAI (cloud)":
        if not openai_api_key:
            st.error("Please enter your OpenAI API key.")
        else:
            messages = [
                {"role": "system", "content": "You are a helpful assistant."},
                {"role": "user", "content": user_input}
            ]
            reply = chat_with_openai(messages, model=openai_model, api_key=openai_api_key)
            st.markdown(f"**Assistant:** {reply}")

```

Running the App

Make sure you have Streamlit installed:

```
pip install streamlit requests openai
```

1.

Start Ollama server in the background:

```
ollama serve
```

```
ollama pull llama3.1:8b
```

```
ollama pull gpt-oss-20b
```

2.

Run the app:

```
streamlit run app.py
```

3.

4. Open the browser window → select **Ollama** or **OpenAI** → enter your message → get a response!

Example Use Cases

Just like the AI use-cases in your PDF (recommendations, sentiment, CSAT prediction), this app can be extended into:

- **Local prototyping** with Ollama before deploying cloud LLMs.
- **Hybrid workflows:** sensitive data handled locally, advanced reasoning offloaded to OpenAI.

- **Teaching/demo apps** for quickly experimenting with LLMs in a user-friendly UI.

Takeaway

With just ~50 lines of code, we combined **local AI** and **cloud AI** into one seamless Streamlit app.

This flexibility is powerful — you control costs, privacy, and model choice while giving end-users a simple interface.

Chat with Ollama or OpenAI

Choose provider

- ☐ Ollama (local)
☒ OpenAI (cloud)

Enter OpenAI API Key

.....



Select OpenAI model

gpt-3.5-turbo



Your message

what is capital of india

Send

Assistant: The capital of India is New Delhi.